Makefile从入门到上手_makefile菜鸟教程-CSDN博客

前言

makefile 关系到了整个工程的编译规则。一个工程中的源文件不计其数，并且按类型、功能、模块分别放在若干个目录中，makefile 定义了一系列的规则来指定，哪些文件需要先编译，哪些文件需要后编译，哪些文件需要重新编译，甚至于进行更复杂的功能操作，因为 makefile 就像一个 Shell 脚本一样，其中也可以执行操作系统的命令。

本文记录了 Makefile 的使用。

一、Makefile 介绍

Makefile 是一个名为 GNU-Make 软件所需要的脚本文件，该脚本文件可以指导 Make 软件控制 arm-gcc 等工具链去编译工程文件最终得到可执行文件，几乎所有的 Linux 发行版都内置了 GNU-Make 软件，VScode 等多种 IDE 也内置了 Make 程序。**说白了，Makefile 就是用来管理项目的。**

你见到的 **xxx.mk** 文件或者 **Makefile** 都统称为 Makefile 脚本文件。

命名只能为 makefile 或者是 Makefile ，因为只有这两种命名方式才能被 make 命令识别。

Makefile 脚本文件的语法学习可以参考： 跟我一起写Makefile 陈皓

二、示例源码

这里放出供下面 **Makefile 基础规则** 中用来测试的源码。

1、hello.c

```
#include <stdio.h>
#include "head.h"

int main(int argc, char *argv[])
{
    int a = 10; int b = 5;
```

```
    printf("%d+%d=%d\n", a, b, add(a, b));
    printf("%d-%d=%d\n", a, b, sub(a, b));
    printf("%d/%d=%d\n", a, b, div(a, b));
    printf("%dx%d=%d\n", a, b, mul(a, b));

    return 0;
}
c1234567891011121314
```

2、add.c

```
int add(int a, int b)
{
    return a+b;
}
c1234
```

3、sub.c

```
int sub(int a, int b)
{
    return a-b;
}
c1234
```

4、mul.c

```
int mul(int a, int b)
{
    return a*b;
}
c1234
```

5、div.c

```
int div(int a, int b)
{
    return a/b;
}
c1234
```

6、head.h

```
#ifndef _HEAD_H_
#define _HEAD_H_

int add(int , int );
int sub(int , int );
int div(int , int );
int mul(int , int );

#endif
c123456789
```

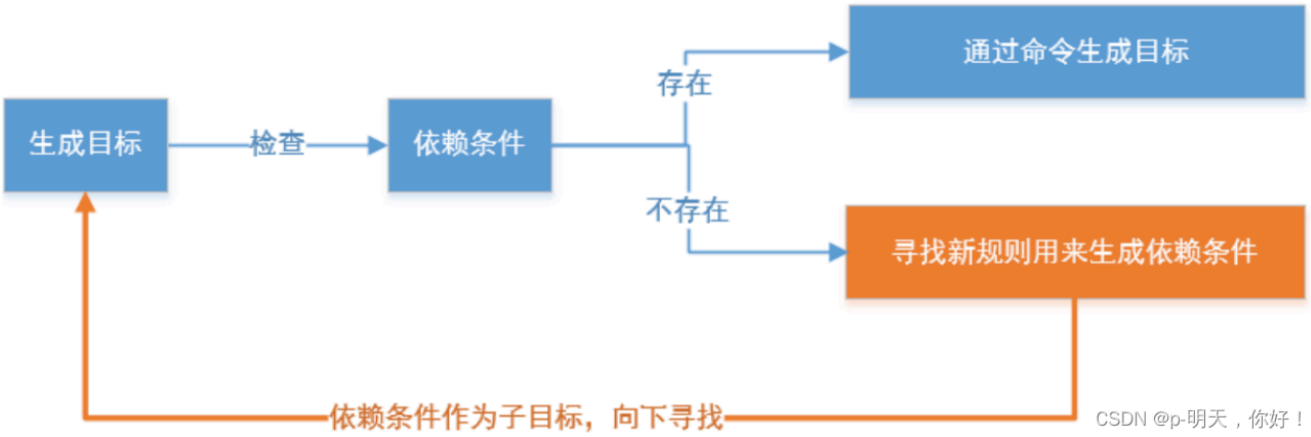
三、Makefile 基础规则

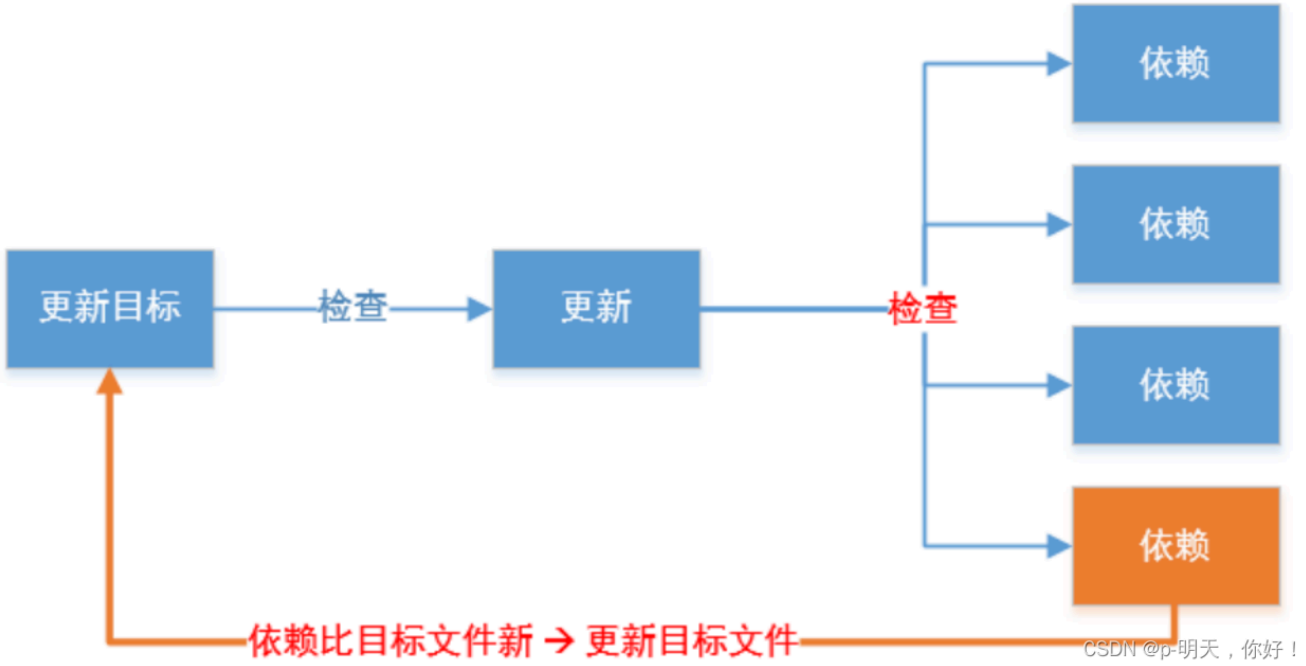
想要掌握 makefile，首先需要了解两个概念，一个是目标（target），另一个就是依赖（dependency） 。目标就是指要干什么，或说运行 make 后生成什么，而依赖是告诉 make 如何去做以实现目标。在 Makefile 中，目标和依赖是通过规则（rule）来表达的。

makefile 的依赖是从上至下的，换句话说就是目标文件是第一句里的目标， 如果不满足执行依赖，就会继续向下执行 。如果满足了生成目标的依赖，就不会再继续向下执行了。make 会自动寻找规则里需要的材料文件，执行规则下面的行为生成规则中的目标。

Makefile 三要素：

工作原理：





1、一个规则

1个规则

```
目标：依赖条件
（一个 tab 缩进）命令
css12
```

①、目标的时间必须晚于依赖条件的时间，否则，更新目标。
②、依赖条件如果不存在，找寻新的规则去产生依赖条件。

下面来一步一步升级 makefile

利用上述代码实现第一个版本的 Makefile

Makefile 第一版

```
a.out : hello.c add.c sub.c mul.c div.c
    gcc hello.c add.c sub.c mul.c div.c -o a.out -I ./
bash12
```

执行 make：

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ make
gcc hello.c add.c sub.c mul.c div.c -o a.out -I ./
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

此时，修改 add.c 为下面

add.c

```
int add(int a, int b)
{
    return a+b+1;
}
c1234
```

此时，再使用 make，发现了问题

```
Gnep@lpvm:~/test/makefile_test$ vi add.c
Gnep@lpvm:~/test/makefile_test$ make
gcc hello.c add.c sub.c mul.c div.c -o a.out -I ./
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=16
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

可以看到，只修改 add.c，但是编译的时候，其他.c 文件也重新编译了，这不太科学。明明只改了一个，全部都重新编译了。

于是将 makefile 改写如下：

Makefile 第二版

```
a.out : hello.o add.o sub.o mul.o div.o
    gcc hello.o add.o sub.o mul.o div.o -o a.out

hello.o : hello.c
    gcc -c hello.c -o hello.o -I ./

add.o : add.c
    gcc -c add.c -o add.o

sub.o : sub.c
    gcc -c sub.c -o sub.o

mul.o : mul.c
    gcc -c mul.c -o mul.o

div.o : div.c
    gcc -c div.c -o div.o
bash123456789101112131415161 7
```

执行 make 指令如下：

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o -I ./
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc -c mul.c -o mul.o
gcc -c div.c -o div.o
gcc hello.o add.o sub.o mul.o div.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=16
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

此时，再将 add.c 改为最开始的代码：

add.c

```
int add(int a, int b)
{
    return a+b;
}
c1234
```

执行 make 指令如下：

```
Gnep@lpvm:~/test/makefile_test$ vi add.c
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc hello.o add.o sub.o mul.o div.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

可以看到，只重新编译了修改过的 add.c 和最终目标

`makefile` 检测原理：修改文件后，文件的修改时间发生变化，会出现目标文件的时间早于作为依赖材料的时间，出现这种情况的文件会重新编译。修改 `add.c` 后， `add.o` 的时间就早于 `add.c` ， `a.out` 的时间也早于 `add.o` 的时间了，于是重新编译这俩文件了。

关于 makefile 指定目标问题，先修改 makefile 如下：

Makefile 第三版

```
hello.o : hello.c
    gcc -c hello.c -o hello.o -I ./

add.o : add.c
    gcc -c add.c -o add.o

sub.o : sub.c
    gcc -c sub.c -o sub.o

mul.o : mul.c
    gcc -c mul.c -o mul.o

div.o : div.c
    gcc -c div.c -o div.o

a.out : hello.o add.o sub.o mul.o div.o
    gcc hello.o add.o sub.o mul.o div.o -o a.out
bash1234567891011121314151617
```

只是将 a.out 放在了文件末尾

执行 make，如下：

```
Gnep@lpvm:~/test/makefile_test$ rm *.o
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o -I ./
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  hello.o  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

这是因为， makefile 默认第一个目标文件为终极目标，生成就跑路，这时候可以用 ALL 来指定终极目标。

```
ALL: 指定 makefile 的终极目标。
bash1
```

指定目标的 makefile 如下：

Makefile 第四版

```
ALL : a.out

hello.o : hello.c
    gcc -c hello.c -o hello.o -I ./

add.o : add.c
    gcc -c add.c -o add.o

sub.o : sub.c
    gcc -c sub.c -o sub.o

mul.o : mul.c
    gcc -c mul.c -o mul.o

div.o : div.c
    gcc -c div.c -o div.o

a.out : hello.o add.o sub.o mul.o div.o
    gcc hello.o add.o sub.o mul.o div.o -o a.out
bash1234567891011121314151617181 9
```

执行：

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o -I ./
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc -c mul.c -o mul.o
gcc -c div.c -o div.o
gcc hello.o add.o sub.o mul.o div.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o   hello.c  Makefile  mul.o  sub.o
add.o  div.c  head.h  hello.o  mul.c     sub.c
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

2、两个函数和 clean

①、2 个函数：

找到当前目录下所有后缀为.c 的文件，赋值给 src

```
src = $(wildcard ./*.c) # 匹配当前工作目录下的所有.c 文件。将文件名组成列表，赋值给变量 src。 src = add.c sub.c mul.c div1.c
bash1
```

把 src 变量所有后缀为.c 的文件替换成.o

```
obj = $(patsubst %.c, %.o, $(src)) # 将参数 3 中，包含参数 1 的部分，替换为参数 2。 obj = add.o sub.o mul.c div1.o
bash1
```

用这两个函数修改 makefile 如下：

Makefile 第五版

```
src = $(wildcard *.c) # hello.c add.c sub.c mul.c  div.c

obj = $(patsubst %.c, %.o, $(src)) # hello.o add.o sub.o mul.o div.o

ALL : a.out

hello.o : hello.c
```

```
    gcc -c hello.c -o hello.o -I ./

add.o : add.c
    gcc -c add.c -o add.o

sub.o : sub.c
    gcc -c sub.c -o sub.o

mul.o : mul.c
    gcc -c mul.c -o mul.o

div.o : div.c
    gcc -c div.c -o div.o

a.out : $(obj)
    gcc $(obj) -o a.out
bash1234567891011121314151617181920212223
```

执行 make 指令，如下所示：

```
Gnep@lpvm:~/test/makefile_test$ rm *.o
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o -I ./
gcc -c mul.c -o mul.o
gcc -c sub.c -o sub.o
gcc  add.o  div.o  hello.o  mul.o  sub.o  -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

②、clean

```
-rm -rf $(obj) a.out # “-” ：作用是，删除不存在文件时，不报错。顺序执行结束。
```

```
bash1
```

每次要删除.o 文件，很恶心，于是改写 makefile 如下：
加了 clean 部分

Makefile 第六版

```
src = $(wildcard *.c) # hello.c add.c sub.c mul.c  div.c

obj = $(patsubst %.c, %.o, $(src)) # hello.o add.o sub.o mul.o div.o

ALL : a.out

hello.o : hello.c
    gcc -c hello.c -o hello.o -I ./

add.o : add.c
    gcc -c add.c -o add.o

sub.o : sub.c
    gcc -c sub.c -o sub.o

mul.o : mul.c
    gcc -c mul.c -o mul.o

div.o : div.c
    gcc -c div.c -o div.o

a.out : $(obj)
    gcc $(obj) -o a.out

clean :
    -rm -rf $(obj) a.out
bash1234567891011121314151617181920212223242526
```

rm 前面的 - ，代表出错依然执行。比如，待删除文件集合是 5 个，已经手动删除了 1 个，就只剩下 4 个，然而删除命令里面还是 5 个的集合，就会有删除不存在文件的问题，不加这 - ，就会报错，告诉你有一个文件找不到。加了-就不会因为这个报错。

执行 make

```
Gnep@lpvm:~/test/makefile_test$ make
make: 对“ALL”无需做任何事。
```

CSDN @p-明天，你好！

由于没有文件变动， a.out 的时间戳晚于所有依赖文件，这里 make 就没干活

于是，执行时加新指令，先模拟执行 clean 部分：

```
Gnep@lpvm:~/test/makefile_test$ make clean -n
rm -rf  add.o  div.o  hello.o  mul.o  sub.o  a.out
```

CSDN @p-明天，你好！

确定没有问题，执行

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o  hello.c  Makefile  mul.o  sub.o
add.o  div.c  head.h  hello.o  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

3、三个自动变量和模式规则

①、三个自动变量

```
$@    # 在规则的命令中，表示规则中的目标。
bash1
```

```
$^    # 在规则的命令中，表示所有依赖条件。组成一个列表，以空格隔开，如果这个列表中有重复项，则去重
bash1
```

```
$<    # 在规则的命令中，表示第一个依赖条件。如果将该变量应用在模式规则中，它可将依赖条件列表中的依赖依次取出，套用模式规则。
bash1
```

用自动变量修改 makefile，如下：

Makefile 第七版

```
src = $(wildcard *.c) # hello.c add.c sub.c mul.c  div.c

obj = $(patsubst %.c, %.o, $(src)) # hello.o add.o sub.o mul.o div.o

ALL : a.out

hello.o : hello.c
    gcc -c $< -o $@ -I ./

add.o : add.c
    gcc -c $< -o $@

sub.o : sub.c
    gcc -c $< -o $@

mul.o : mul.c
    gcc -c $< -o $@

div.o : div.c
    gcc -c $< -o $@

a.out : $(obj)
    gcc $^ -o $@

clean :
    -rm -rf $(obj) a.out
bash1234567891011121314151617181920212223242526
```

sub， add 这些指令中使用 $< 和 $^ 都能达到效果，但是为了模式规则，所以使用的 $<

执行 make，如下：

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o -I ./
gcc -c mul.c -o mul.o
gcc -c sub.c -o sub.o
gcc add.o div.o hello.o mul.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

再来，**上面这个 makefile，可扩展性不行。**比如，要添加一个平方函数，就需要在 makefile 里面增加平方函数的部分。 **不科学， 所以，模式规则就来了** 。

②、模式规则

<1>、模式规则

```
%.o : %.c
    gcc -c $< -o %@
bash12
```

修改 makefile，如下：

Makefile 第八版

```
src = $(wildcard *.c) # hello.c add.c sub.c mul.c  div.c

obj = $(patsubst %.c, %.o, $(src)) # hello.o add.o sub.o mul.o div.o

ALL : a.out

%.o : %.c
    gcc -c $< -o $@
```

```
a.out : $(obj)
    gcc $^ -o $@

clean :
    -rm -rf $(obj) a.out
bash1234567891011121314
```

执行 make，如下：

```
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o
gcc -c mul.c -o mul.o
gcc -c sub.c -o sub.o
gcc add.o div.o hello.o mul.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o  hello.c  Makefile  mul.o  sub.o
add.o  div.c  head.h  hello.o  mul.c     sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

这时，增加一个 square 函数， 并添加 square.c 文件如下：

square.c

```
int suqare(int a)
{
    return a*a;
}
c1234
```

head.h 做如下修改：

head.h

```
#ifndef _HEAD_H_
#define _HEAD_H_

int add(int , int );
int sub(int , int );
```

```
int div(int , int );
int mul(int , int );
int square(int );

#endif
c12345678910
```

hello.c 做如下修改：

hello.c

```
#include <stdio.h>
#include "head.h"

int main(int argc, char *argv[])
{
    int a = 10; int b = 5;

    printf("%d+%d=%d\n", a, b, add(a, b));
    printf("%d-%d=%d\n", a, b, sub(a, b));
    printf("%d/%d=%d\n", a, b, div(a, b));
    printf("%dx%d=%d\n", a, b, mul(a, b));
    printf("%d*%d=%d\n", a, a, mul(a, a));

    return 0;
}
c123456789101112131415
```

直接执行 make：

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o   hello.c  Makefile  mul.o     sub.c
add.o  div.c  head.h  hello.o  mul.c     square.c  sub.o
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o
gcc -c square.c -o square.o
gcc add.o div.o hello.o mul.o square.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

增加函数的时候，不用改 makefile，只需要增加.c 文件，改一下源码，就行。很强势。

<2>、静态模式规则

继续优化 makefile， **使用静态模式规则，就是指定模式规则给谁用** ，这里指定模式规则给 obj 用，以后文件多了，文件集合会有很多个，就需要指定哪个文件集合用什么规则

```
$(obj) : %.o : %.c
    gcc -c $< -o %@
bash12
```

修改后 makefile 如下：

Makefile 第八版

```
src = $(wildcard *.c) # hello.c add.c sub.c mul.c  div.c

obj = $(patsubst %.c, %.o, $(src)) # hello.o add.o sub.o mul.o div.o

ALL : a.out

$(obj) : %.o : %.c
    gcc -c $< -o $@

a.out : $(obj)
    gcc $^ -o $@

clean :
    -rm -rf $(obj) a.out
bash1234567891011121314
```

运行如下：

```
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  square.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o
gcc -c mul.c -o mul.o
gcc -c square.c -o square.o
gcc -c sub.c -o sub.o
gcc add.o div.o hello.o mul.o square.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

4、伪目标

```
.PHONY: clean ALL
bash1
```

再来一个扩展，当前文件夹下有 ALL 文件或者 clean 文件时，会导致 makefile 瘫痪，如下所示， make clean 没有工作

```
Gnep@lpvm:~/test/makefile_test$ touch clean
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.c  head.h  hello.o  mul.c  square.c  sub.c
add.o  clean  div.o  hello.c  Makefile  mul.o  square.o  sub.o
Gnep@lpvm:~/test/makefile_test$ make clean
make: “clean”已是最新。
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

用伪目标来解决， 添加一行.PHONY: clean ALL

makefile 如下所示：

Makefile 第九版

```
src = $(wildcard *.c) # hello.c add.c sub.c mul.c  div.c

obj = $(patsubst %.c, %.o, $(src)) # hello.o add.o sub.o mul.o div.o

ALL : a.out

$(obj) : %.o : %.c
    gcc -c $< -o $@

a.out : $(obj)
    gcc $^ -o $@

clean :
    -rm -rf $(obj) a.out

.PHONY : clean ALL
bash123456789101112131415 16
```

再来执行 make clean，就不会受到干扰了

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.c  head.h  hello.o  mul.c  square.c  sub.c
add.o  clean  div.o  hello.c  Makefile  mul.o  square.o  sub.o
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  square.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  clean  div.c  head.h  hello.c  Makefile  mul.c  square.c  sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

5、编译时的参数

还有一个扩展就是，编译时的参数， -g，-Wall 这些，可以放在 makefile 里面 修改后 makefile 如下：

Makefile 第十版

```
src = $(wildcard *.c) # hello.c add.c sub.c mul.c  div.c

obj = $(patsubst %.c, %.o, $(src)) # hello.o add.o sub.o mul.o div.o

myArgs = -Wall -g

ALL : a.out

$(obj) : %.o : %.c
```

```
    gcc -c $< -o $@ $(myArgs)

a.out : $(obj)
    gcc $^ -o $@ $(myArgs)

clean :
    -rm -rf $(obj) a.out

.PHONY : clean ALL
bash123456789101112131415161718
```

执行 make，如下：

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o -Wall -g
gcc -c div.c -o div.o -Wall -g
gcc -c hello.c -o hello.o -Wall -g
gcc -c mul.c -o mul.o -Wall -g
gcc -c square.c -o square.o -Wall -g
gcc -c sub.c -o sub.o -Wall -g
gcc add.o div.o hello.o mul.o square.o sub.o -o a.out -Wall -g
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

6、make 的参数

参数：

- -n：模拟执行 make、 make clean 命令。
- -f：指定文件执行 make 命令。 xxxx.mk

如果 makefile 的名字变化一下，比如，叫 m6
用 m6 执行 makefile， make -f m6
用 m6 执行 clean make -f m6 clean

四、工程源码优化

将上述.c 文件都放到 src 目录中，.h 文件都放在 inc 目录中，所生成的.o 文件产物都放在 obj 目录中

```
Gnep@lpvm:~/test/makefile_test$ mkdir inc
Gnep@lpvm:~/test/makefile_test$ mkdir obj
Gnep@lpvm:~/test/makefile_test$ mkdir src
Gnep@lpvm:~/test/makefile_test$ mv *.c src/
Gnep@lpvm:~/test/makefile_test$ mv *h inc/
Gnep@lpvm:~/test/makefile_test$ ls
clean  inc  Makefile  obj  src
```

CSDN @p-明天，你好！

使用 tree 命令查看树形结构拓扑

```
Gnep@lpvm:~/test/makefile_test$ tree
.
├── clean
├── inc
│   └── head.h
├── Makefile
├── obj
└── src
    ├── add.c
    ├── div.c
    ├── hello.c
    ├── mul.c
    ├── square.c
    └── sub.c

3 directories, 9 files
```

CSDN @p-明天，你好！

修改 makefile 如下，主要是注意 % 的匹配理解，只匹配文件名，目录位置要手动添加

Makefile 第十一版

```
src = $(wildcard ./src/*.c) # ./src/hello.c ./src/add.c ...

obj = $(patsubst ./src/%.c, ./obj/%.o, $(src)) # ./obj/hello.o
./obj/add.o ...

inc_path = ./inc

myArgs = -Wall -g

ALL : a.out

$(obj) : ./obj/%.o : ./src/%.c
    gcc -c $< -o $@ $(myArgs) -I $(inc_path)

a.out : $(obj)
    gcc $^ -o $@ $(myArgs)

clean :
    -rm -rf $(obj) a.out

.PHONY : clean ALL
bash1234567891011121314151617181920
```

执行 make 和 make clean，效果如下

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c src/add.c -o obj/add.o -Wall -g -I ./inc
gcc -c src/div.c -o obj/div.o -Wall -g -I ./inc
gcc -c src/hello.c -o obj/hello.o -Wall -g -I ./inc
gcc -c src/mul.c -o obj/mul.o -Wall -g -I ./inc
gcc -c src/square.c -o obj/square.o -Wall -g -I ./inc
gcc -c src/sub.c -o obj/sub.o -Wall -g -I ./inc
gcc obj/add.o obj/div.o obj/hello.o obj/mul.o obj/square.o obj/sub.o -o a.out -Wall -g
Gnep@lpvm:~/test/makefile_test$ ls
a.out  inc  Makefile  obj  src
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  ./obj/add.o  ./obj/div.o  ./obj/hello.o  ./obj/mul.o  ./obj/square.o  ./obj/sub.o
a.out
Gnep@lpvm:~/test/makefile_test$ ls
inc  Makefile  obj  src
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

如果 makefile 的名字变化一下，比如，叫 m6

用 m6 执行 makefile， make -f m6

用 m6 执行 clean make -f m6 clean

```
Gnep@lpvm:~/test/makefile_test$ ls
inc  m6  obj  src
Gnep@lpvm:~/test/makefile_test$ make -f m6
gcc -c src/add.c -o obj/add.o -Wall -g -I ./inc
gcc -c src/div.c -o obj/div.o -Wall -g -I ./inc
gcc -c src/hello.c -o obj/hello.o -Wall -g -I ./inc
gcc -c src/mul.c -o obj/mul.o -Wall -g -I ./inc
gcc -c src/square.c -o obj/square.o -Wall -g -I ./inc
gcc -c src/sub.c -o obj/sub.o -Wall -g -I ./inc
gcc obj/add.o obj/div.o obj/hello.o obj/mul.o obj/square.o obj/sub.o -o a.out -Wall -g
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$ make -f m6 clean
rm -rf  ./obj/add.o  ./obj/div.o  ./obj/hello.o  ./obj/mul.o  ./obj/square.o  ./obj/sub.o
a.out
Gnep@lpvm:~/test/makefile_test$ ls
inc  m6  obj  src
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

五、Makefile 语法

1、两种变量

在 Makefile 中有两种变量，一种称为即时变量（简单变量），另一种称为延时变量

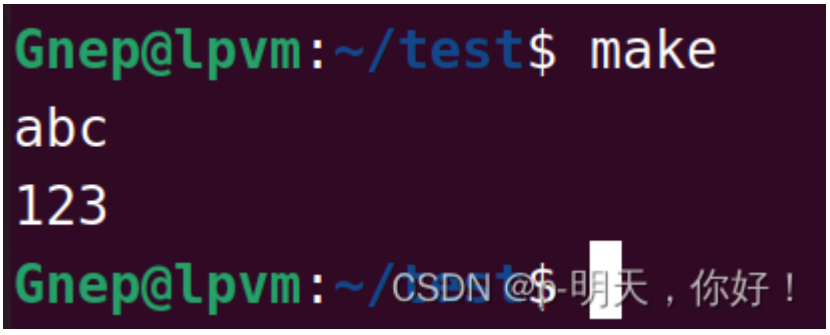
- 即时变量（简单变量）
  - `A := xxx # A 的值即刻确定，在定义时即确定`
- 延时变量
  - `B = xxx # B 的值使用到时才确定`

编写下面一个 Makefile

```
A := abc
B = 123

all :
    @echo $(A)
    @echo $(B)
bash123456
```

执行 make



从上面我们看不出即时变量和延时变量的差别，我们再对 Makefile 进行如下修改：

```
A := $(C)
B = $(C)
C = abc

all :
    @echo A = $(A)
    @echo B = $(B)
bash1234567
```

执行 make

```
Gnep@lpvm:~/test$ make
A =
B = abc
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

可以看到 A 的值为空，B 的值为 abc，因为 A 为即时变量，在定义时即确定，所以为空

修改 Makefile 将 C 的赋值放在最后：

```
A := $(C)
B = $(C)
#C = abc

all :
    @echo A = $(A)
    @echo B = $(B)

C = abc
bash123456789
```

执行 make，可以发现结果并不受影响

```
Gnep@lpvm:~/test$ make
A =
B = abc
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

因为当我们执行 make 的时候，会把 Makefile 整个文件读进来进行分析，然后解析里面的变量，所以变量 C 的赋值放在哪里并不受影响

2、赋值方法

```
:=          # 即时变量
=           # 延时变量
?=          # 延时变量，如果是第 1 次定义才起效，如果在前面该变量已定义则忽略这句
+=          # 附加，他是即时变量还是延时变量取决于前面的定义
bash1234
```

①、+=（附加） 使用案例

如下 Makefile

```
A := $(C)
B = $(C)
C = 123

all :
    @echo A = $(A)
    @echo B = $(B)

C += abc
bash123456789
```

执行 make

```
Gnep@lpvm:~/test$ make
A =
B = 123 abc
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

②、?= 使用案例

修改 Makefile 如下：

```
A := $(C)
B = $(C)
C = 123

D = 777
D ?= 888

all :
    @echo A = $(A)
    @echo B = $(B)
    @echo D = $(D)

C += abc
bash12345678910111213
```

执行 make

```
Gnep@lpvm:~/test$ make
A =
B = 123 abc
D = 777
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

六、Makefile 函数

函数调用，很像变量的使用，也是以 $ 来标识的，其语法如下：

```
$(<function> <arguments>)
bash1
```

或是:

```
${<function> <arguments>}
bash1
```

这里， 就是函数名，make支持的函数不多。 为函数的参数，参数间以逗号, 分隔，而函数名和参数之间以“空格”分隔。函数调用以 $ 开头，以圆括号或花括号把函数名和参数括起。

1、foreach 函数

foreach 函数和别的函数非常的不一样。因为这个函数是用来做循环用的，Makefile 中的 foreach 函数几乎是仿照于 Unix 标准 Shell（/bin/sh）中的 for 语句，或是 C-Shell（/bin/csh）中的 foreach 语句而构建的。它的语法是：

```
$(foreach <var>,<list>,<text>)
bash1
```

这个函数的意思是，把参数 中的单词逐一取出放到参数 *所指定的变量中，然后再执行*

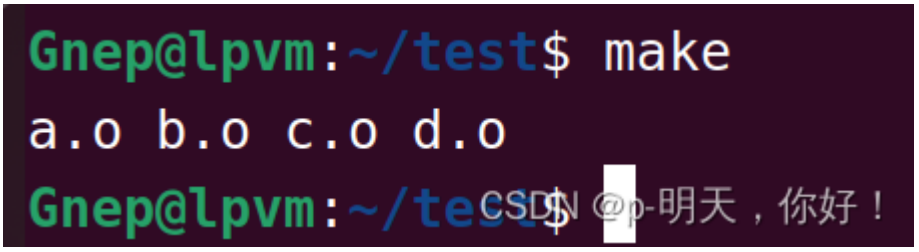
所以， *最好是一个变量名， 可以是一个表达式，而*

```
names := a b c d
```

```
files := $(foreach n,$(names),$(n).o)

ALL:
    @echo $(files)
bash123456
```

执行 make



上面的例子中， $(name) 中的单词会被挨个取出，并存到变量 n 中， $(n).o 每次根据 $(n) 计算出一个值，这些值以空格分隔，最后作为foreach函数的返回，所以，$(files) 的值是 a.o b.o c.o d.o 。

注意，foreach中的 *参数是一个临时的局部变量，foreach函数执行完后，参数 的变量将不在作用，其作用域只在 foreach 函数当中。*

2、filter 和 filter-out 函数

```
$(filter <pattern...>,<text>)
bash1
```

- 名称：过滤函数
- 功能：以
- 返回：返回符合模式

```
$(filter-out <pattern...>,<text>)
bash1
```

- 名称：反过滤函数
- 功能：以
- 返回：返回不符合模式

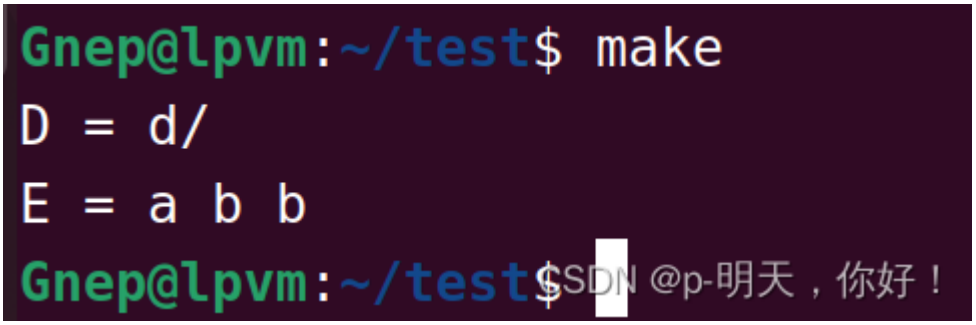
示例 Makefile 如下：

```
C = a b b d/

D = $(filter %/, $(C))
E = $(filter-out %/, $(C))
```

```
ALL:
    @echo D = $(D)
    @echo E = $(E)
bash12345678
```

执行 make



3、patsubst

```
$(patsubst <pattern>,<replacement>,<text>)
bash1
```

- 名称：模式字符串替换函数。
- 功能：查找
- 返回：函数返回被替换过后的字符串。
- 示例： `$(patsubst %.c,%.o,x.c.c bar.c)` ，把字串 x.c.c bar.c 符合模式 %.c 的单词替换成 %.o ，返回结果是 x.c.o bar.o

---

我的qq：2442391036，欢迎交流！

打赏作者

¥1 ¥2 ¥4 ¥6 ¥10 ¥20

扫码支付： ¥1

获取中

扫码支付

您的余额不足，请更换扫码支付或 充值

打赏作者

实付 元

使用余额支付

点击重新获取

扫码支付

钱包余额 0

抵扣说明:

1.余额是钱包充值的虚拟货币，按照1:1的比例进行支付金额的抵扣。
2.余额无法直接购买下载，可以购买VIP、付费专栏及课程。

余额充值

举报

AI 搜索 智能体 AI 编程 AI 作业助手

隐藏侧栏

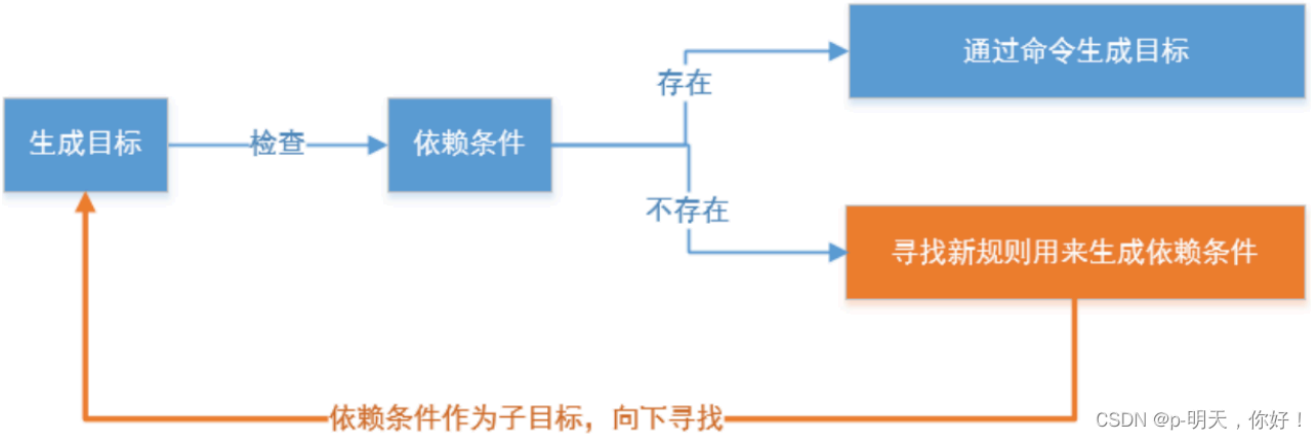
程序员都在用的中文IT技术交流社区

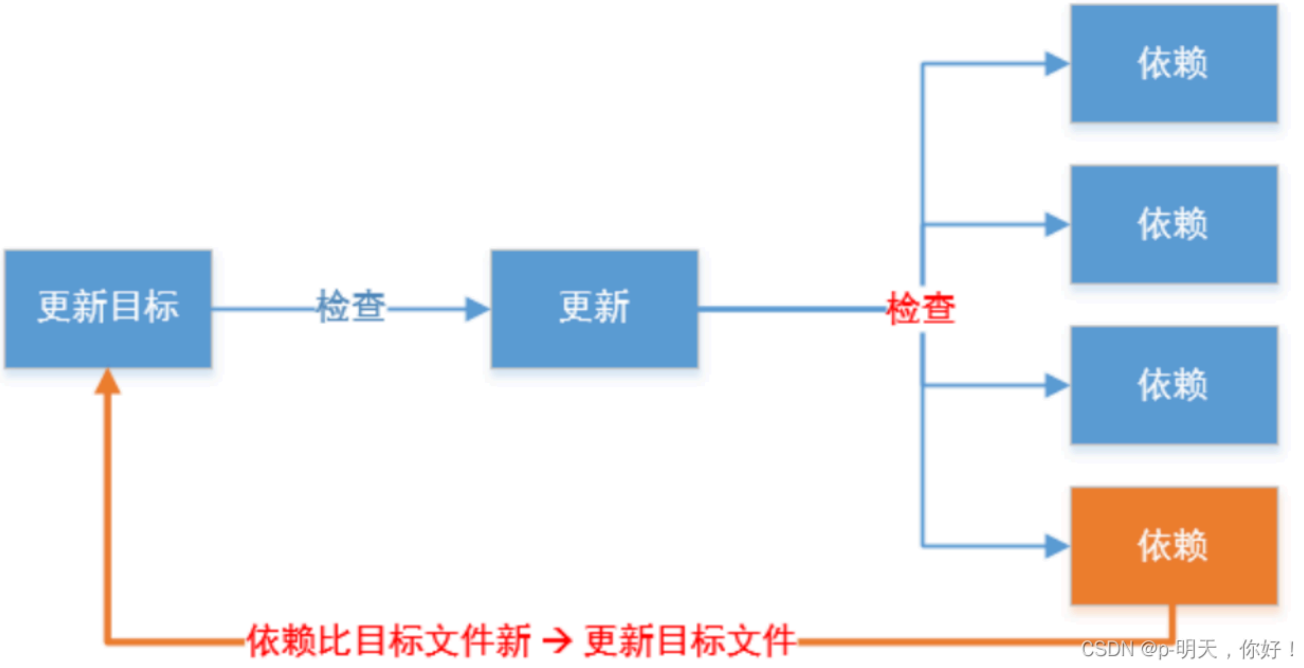
专业的中文 IT 技术社区，与千万技术人共成长

关注【CSDN】视频号，行业资讯、技术分享精彩不断，直播好礼送不停！

客服 返回顶部

依赖
add.c sub.c main.c
目标
main
命令
gcc add.c sub.c main.c -o main
CSDN @p-明天，你好！


生成目标
检查
依赖条件
存在
通过命令生成目标
不存在
寻找新规则用来生成依赖条件
依赖条件作为子目标，向下寻找
CSDN @p-明天，你好！


更新目标
检查
更新
检查
依赖
依赖
依赖
依赖
依赖比目标文件新 → 更新目标文件
CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ make
gcc hello.c add.c sub.c mul.c div.c -o a.out -I ./
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ vi add.c
Gnep@lpvm:~/test/makefile_test$ make
gcc hello.c add.c sub.c mul.c div.c -o a.out -I ./
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=16
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o -I ./
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc -c mul.c -o mul.o
gcc -c div.c -o div.o
gcc hello.o add.o sub.o mul.o div.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=16
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ vi add.c
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc hello.o add.o sub.o mul.o div.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ rm *.o
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o -I ./
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  hello.o  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o -I ./
gcc -c add.c -o add.o
gcc -c sub.c -o sub.o
gcc -c mul.c -o mul.o
gcc -c div.c -o div.o
gcc hello.o add.o sub.o mul.o div.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o   hello.c  Makefile  mul.o  sub.o
add.o  div.c  head.h  hello.o  mul.c     sub.c
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ rm *.o
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o -I ./
gcc -c mul.c -o mul.o
gcc -c sub.c -o sub.o
gcc  add.o  div.o  hello.o  mul.o  sub.o  -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make
make: 对“ALL”无需做任何事。
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make clean -n
rm -rf  add.o  div.o  hello.o  mul.o  sub.o  a.out
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o  hello.c  Makefile  mul.o  sub.o
add.o  div.c  head.h  hello.o  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  div.c  head.h  hello.c  Makefile  mul.c  sub.c
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o -I ./
gcc -c mul.c -o mul.o
gcc -c sub.c -o sub.o
gcc add.o div.o hello.o mul.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o
gcc -c mul.c -o mul.o
gcc -c sub.c -o sub.o
gcc add.o div.o hello.o mul.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o   hello.c  Makefile  mul.o  sub.o
add.o  div.c  head.h  hello.o  mul.c     sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.o  hello.c  Makefile  mul.o  sub.c
add.o  div.c  head.h  hello.o  mul.c  square.c  sub.o
Gnep@lpvm:~/test/makefile_test$ make
gcc -c hello.c -o hello.o
gcc -c square.c -o square.o
gcc add.o div.o hello.o mul.o square.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  square.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o
gcc -c div.c -o div.o
gcc -c hello.c -o hello.o
gcc -c mul.c -o mul.o
gcc -c square.c -o square.o
gcc -c sub.c -o sub.o
gcc add.o div.o hello.o mul.o square.o sub.o -o a.out
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ touch clean
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.c  head.h  hello.o  mul.c  square.c  sub.c
add.o  clean  div.o  hello.c  Makefile  mul.o  square.o  sub.o
Gnep@lpvm:~/test/makefile_test$ make clean
make: “clean”已是最新。
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ ls
add.c  a.out  div.c  head.h  hello.o  mul.c  square.c  sub.c
add.o  clean  div.o  hello.c  Makefile  mul.o  square.o  sub.o
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  add.o  div.o  hello.o  mul.o  square.o  sub.o  a.out
Gnep@lpvm:~/test/makefile_test$ ls
add.c  clean  div.c  head.h  hello.c  Makefile  mul.c  square.c  sub.c
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c add.c -o add.o -Wall -g
gcc -c div.c -o div.o -Wall -g
gcc -c hello.c -o hello.o -Wall -g
gcc -c mul.c -o mul.o -Wall -g
gcc -c square.c -o square.o -Wall -g
gcc -c sub.c -o sub.o -Wall -g
gcc add.o div.o hello.o mul.o square.o sub.o -o a.out -Wall -g
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ mkdir inc
Gnep@lpvm:~/test/makefile_test$ mkdir obj
Gnep@lpvm:~/test/makefile_test$ mkdir src
Gnep@lpvm:~/test/makefile_test$ mv *.c src/
Gnep@lpvm:~/test/makefile_test$ mv *h inc/
Gnep@lpvm:~/test/makefile_test$ ls
clean  inc  Makefile  obj  src
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ tree
.
├── clean
├── inc
│   └── head.h
├── Makefile
├── obj
└── src
    ├── add.c
    ├── div.c
    ├── hello.c
    ├── mul.c
    ├── square.c
    └── sub.c

3 directories, 9 files
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ make
gcc -c src/add.c -o obj/add.o -Wall -g -I ./inc
gcc -c src/div.c -o obj/div.o -Wall -g -I ./inc
gcc -c src/hello.c -o obj/hello.o -Wall -g -I ./inc
gcc -c src/mul.c -o obj/mul.o -Wall -g -I ./inc
gcc -c src/square.c -o obj/square.o -Wall -g -I ./inc
gcc -c src/sub.c -o obj/sub.o -Wall -g -I ./inc
gcc obj/add.o obj/div.o obj/hello.o obj/mul.o obj/square.o obj/sub.o -o a.out -Wall -g
Gnep@lpvm:~/test/makefile_test$ ls
a.out  inc  Makefile  obj  src
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$ make clean
rm -rf  ./obj/add.o  ./obj/div.o  ./obj/hello.o  ./obj/mul.o  ./obj/square.o  ./obj/sub.o
a.out
Gnep@lpvm:~/test/makefile_test$ ls
inc  Makefile  obj  src
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test/makefile_test$ ls
inc  m6  obj  src
Gnep@lpvm:~/test/makefile_test$ make -f m6
gcc -c src/add.c -o obj/add.o -Wall -g -I ./inc
gcc -c src/div.c -o obj/div.o -Wall -g -I ./inc
gcc -c src/hello.c -o obj/hello.o -Wall -g -I ./inc
gcc -c src/mul.c -o obj/mul.o -Wall -g -I ./inc
gcc -c src/square.c -o obj/square.o -Wall -g -I ./inc
gcc -c src/sub.c -o obj/sub.o -Wall -g -I ./inc
gcc obj/add.o obj/div.o obj/hello.o obj/mul.o obj/square.o obj/sub.o -o a.out -Wall -g
Gnep@lpvm:~/test/makefile_test$ ./a.out
10+5=15
10-5=5
10/5=2
10x5=50
10*10=100
Gnep@lpvm:~/test/makefile_test$ make -f m6 clean
rm -rf  ./obj/add.o  ./obj/div.o  ./obj/hello.o  ./obj/mul.o  ./obj/square.o  ./obj/sub.o
a.out
Gnep@lpvm:~/test/makefile_test$ ls
inc  m6  obj  src
Gnep@lpvm:~/test/makefile_test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test$ make
abc
123
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test$ make
A =
B = abc
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test$ make
A =
B = abc
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test$ make
A =
B = 123 abc
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test$ make
A =
B = 123 abc
D = 777
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test$ make
a.o b.o c.o d.o
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！

```
Gnep@lpvm:~/test$ make
D = d/
E = a b b
Gnep@lpvm:~/test$
```

CSDN @p-明天，你好！